Computer Networks

ASSIGNMENT 1

CO1: *Apply the functions of the OSI layers in enabling reliable data transmission across networks – uses of networks, network hardware and software, components, types of connections, topologies, protocols and standards, the ISO/OSI reference model, transmission media, wireless communication, Ethernet configuration, an introduction to 5G networks, and error detection, error correction and data-link control.*

Assignment Title: Make the Data of a Smart Manufacturing Plant Arrive Correctly and On Time (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task asks you to calculate or analyse first, and then judge, decide or justify)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA07-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Python programs, Packet Tracer (.pkt) files, device configurations, Wireshark captures, command outputs and screenshots.
- Include a README.md with your name, register number, the operating system, Python version and Packet Tracer (or NS-3) version used, the steps to run each program or open each simulation, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given program or device configuration into your repository, replace every TODO with working code or commands without changing the rest, run it (Python 3 or Cisco Packet Tracer) and compare your output with the expected output or result.

Alternative Concept Analysis: Describe how the solution would change if the alternative concept were used, and evaluate both solutions using network measures such as throughput, delay, jitter, packet loss, overhead, reliability, security and cost, supported by your own calculations, simulations or measurements wherever possible.

Scenario — Network for a Smart Manufacturing Plant

An automobile-parts company is building a smart plant. Robots on the shop floor must receive control commands within 10 milliseconds; hundreds of sensors report temperature and vibration; CCTV cameras stream video; office staff use the ERP system and e-mail; and the plant exchanges production data with the head office 400 km away.

The contractor proposes ordinary office Wi-Fi throughout the plant and a single broadband connection to the head office. The plant engineers are worried that electrical noise from welding machines will corrupt data, that robot commands will be delayed, and that the head-office link will be too slow.

The network may grow:

1 production line (40 devices) → 1 plant (1,500 devices) → 5 plants connected to the head office

Each task below gives you data from the plant. Use it to calculate, compare and decide – a correct answer must be supported by your working, not only stated.

Design Challenge:

The company plans to use a private 5G network on the shop floor and to connect autonomous guided vehicles (AGVs).

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Size the Traffic Before Choosing Anything **4 Marks**

Bloom's Level: K5 – Evaluate

One production line generates the following traffic:

Source	Quantity	Traffic
Robot controllers	40	200-byte command every 2 ms each
Vibration sensors	600	50 bytes per second each
CCTV cameras	24	4 Mbps each (continuous)
Office users	150	2 Mbps each at peak, 10% active at a time

Work out:

- The total traffic of one line in Mbps, and of the plant if there are 8 lines
- Which source sends the most data and which has the strictest delay requirement – are they the same?
- Whether a single 1 Gbps uplink from the shop floor is enough, and what happens to robot commands if the CCTV traffic shares the same queue
- A ranking of the traffic types by how harmful a 100 ms delay would be, with a reason for each

Alternative Concept Analysis

Suppose the CCTV cameras recorded locally and sent only motion-triggered clips (about 5% of the time) instead of streaming continuously. Recalculate the line traffic, and decide whether the 1 Gbps uplink is still the bottleneck.

Evidence Expected

Complete calculations with units, a delay-versus-volume comparison and a justified ranking.

TASK 2 – Count the Domains in the Contractor's Drawing — 3 Marks

Bloom's Level: K5 – Evaluate

The contractor's drawing for the office block shows: three hubs with 8 PCs each, all three hubs connected to one 24-port switch, the switch connected to router interface G0/0, and router interface G0/1 connected to a second switch with 12 IP phones.

Work out:

- The number of collision domains and broadcast domains, with a marked drawing
- The OSI layer at which each device works, and why a hub cannot stop collisions
- Which connections are point-to-point and which are multipoint
- Which standard and which body defines each of: Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), IPv4 (IETF RFC 791), the OSI model (ISO 7498), and 5G (3GPP/ITU) – and why the plant should buy only standard-compliant equipment

Alternative Concept Analysis

A colleague proposes replacing the three hubs with three cheap unmanaged switches. Recount the collision and broadcast domains, and state what this change fixes and what it does not.

Evidence Expected

Correct domain counts on a marked drawing, device-layer reasoning and a completed standards table.

TASK 3 – Read a Captured Frame Layer by Layer — 4 Marks

Bloom's Level: K5 – Evaluate

A capture of one robot command shows the following fields:

```
Ethernet II  Dst: 00:1d:9c:c7:b0:41   Src: 3c:52:82:4e:11:07   Type: 0x0800
IPv4         Src: 10.10.1.25  Dst: 10.10.9.4  TTL: 61  Protocol: 6 (TCP)
TCP          Src Port: 51522  Dst Port: 502 (Modbus)  Seq: 1201  Ack: 88  Len: 12
Data         12 bytes (Modbus write-register command)
```

Work out:

- The OSI layer and the protocol data unit (frame, packet, segment, data) of each line
- How many routers the packet crossed if the controller sets TTL to 64
- The total bytes on the wire (include the 4-byte FCS) and the percentage that is overhead; then repeat for a 1,400-byte camera packet – what does this tell you about small commands?
- Which layer is responsible when (a) the MAC address is wrong, (b) the TTL reaches 0, (c) the destination port is closed
- How the TCP/IP model maps onto these seven layers

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""How data is wrapped by each layer at the sender and unwrapped at the receiver."""

LAYERS = [("Application", "HTTP"), ("Transport", "TCP"), ("Network", "IP"),
          ("Data link", "ETH")]



def send(message):
    pdu = message
    for layer, tag in LAYERS:
        # TODO 1: Wrap pdu with this layer's header "[tag|" and, for the data link layer
        #   only, also a trailer "|FCS]"; for other layers close with "]". Print the
        #   layer name and the size of the new PDU
        pass  # replace this line with your code
    return pdu



def receive(frame):
    pdu = frame
    for layer, tag in reversed(LAYERS):
        head = "[" + tag + "|"
        tail = "|FCS]" if tag == "ETH" else "]"
        assert pdu.startswith(head) and pdu.endswith(tail), f"{layer}: bad header"
        pdu = pdu[len(head):-len(tail)]
    return pdu



frame = send("GET /results")
print("on the wire:", frame)
print("delivered  :", receive(frame))
print(f"overhead   : {len(frame) - len('GET /results')} bytes for 12 bytes of data")
```

Expected output:

```
Application  adds HTTP ->  19 bytes
Transport    adds TCP  ->  25 bytes
Network      adds IP   ->  30 bytes
Data link    adds ETH  ->  40 bytes
on the wire: [ETH|[IP|[TCP|[HTTP|GET /results]]]|FCS]
delivered  : GET /results
overhead   : 28 bytes for 12 bytes of data
```

Change the message to 1,400 bytes and compare the overhead percentage with the 12-byte message.

Alternative Concept Analysis

An industrial protocol could carry the robot command directly inside an Ethernet frame (EtherType 0x8892) with no IP or TCP header. Calculate the bytes saved per command and the commands per second this frees on a 100 Mbps link, and identify what the plant loses by giving up IP.

Evidence Expected

A layer-by-layer reading of the capture, correct overhead calculations and a fault-to-layer mapping.

TASK 4 – Choose a Medium for Each Run (There Is a Trap) 4 Marks

Bloom's Level: K5 – Evaluate

The plant has four cable runs: (a) a robot cell next to arc-welding machines, 85 m; (b) the office floor to the shop-floor switch, 140 m; (c) the shop floor to the data centre, 550 m; (d) the plant to the head office, 400 km.

Work out:

- A medium for each run (Cat6 UTP, Cat6A S/FTP, multimode fibre, single-mode fibre, leased line or wireless) with the deciding reason
- Which run cannot use copper Ethernet at all, and why (look at the distance limits)
- The Shannon capacity of a 250 MHz cable with an SNR of 40 dB and of the same cable near the welders with an SNR of 15 dB – what does the difference mean for the robot cell?
- Why the speed measured on a link is always below the Shannon limit

Complete the Code

Complete TODO 1 and TODO 2 below, and run the program with Python 3.

```
"""Nyquist and Shannon limits for the transmission media proposed for a campus."""
import math


def nyquist(bandwidth_hz, levels):
    # TODO 1: Return the Nyquist maximum bit rate for a noiseless channel: 2 x bandwidth
    #   x log2(levels)
    pass  # replace this line with your code


def shannon(bandwidth_hz, snr_db):
    # TODO 2: Convert the SNR from decibels to a ratio (10 ** (dB / 10)) and return the
    #   Shannon capacity: bandwidth x log2(1 + SNR)
    pass  # replace this line with your code


media = [("Telephone line", 3_100, 30), ("Cat6 pair", 250e6, 40),
         ("Wi-Fi 6 channel", 160e6, 25)]
for name, bw, snr in media:
    print(f"{name:<16} Shannon limit {shannon(bw, snr) / 1e6:10.2f} Mbps")
print(f"Nyquist, 3,100 Hz with 4 levels: {nyquist(3_100, 4) / 1e3:.1f} kbps")
```

Expected output:

```
Telephone line   Shannon limit       0.03 Mbps
Cat6 pair        Shannon limit    3321.96 Mbps
Wi-Fi 6 channel  Shannon limit    1329.50 Mbps
Nyquist, 3,100 Hz with 4 levels: 12.4 kbps
```

Find the SNR (in dB) a 3,100 Hz telephone line would need to carry 56 kbps, and judge whether that is realistic.

Alternative Concept Analysis

What if the robot cell used Wi-Fi 6 instead of shielded cable? Using the welding-area SNR of 15 dB on a 160 MHz channel, calculate the Shannon limit, and decide whether a wireless link is acceptable for 10 ms robot commands.

Evidence Expected

A justified medium for every run, the trapped run identified, and Shannon calculations with interpretation.

TASK 5 – Wi-Fi 6 or 5G for Moving Equipment? **4 Marks**

Bloom's Level: K5 – Evaluate

An AGV travels at 2 m/s along a 1.2 km route. Wi-Fi access points are 30 m apart, and a Wi-Fi hand-over takes about 150 ms. A private 5G cell covers 300 m, and a 5G hand-over takes about 20 ms. AGV commands have a 10 ms deadline.

Work out:

- The number of hand-overs per trip for Wi-Fi and for 5G, and the total time per trip during which the AGV is not reachable
- Which of the three 5G service types (eMBB, URLLC, mMTC) the AGVs, the cameras and the sensors each need
- Where Bluetooth, Zigbee or LoRa would be a better choice than Wi-Fi or 5G in the plant
- Whether Wi-Fi 6 with fast roaming (802.11r) could meet the deadline – and what remains risky

Alternative Concept Analysis

The sensors could use LoRaWAN (a few kbps, kilometres of range, years on a battery) instead of Wi-Fi. Calculate whether 600 sensors at 50 bytes per second fit within one LoRa gateway's capacity of roughly 5 kbps, and decide which sensors should stay on Wi-Fi.

Evidence Expected

Hand-over calculations, a mapping of traffic to 5G service types and a justified wireless decision.

TASK 6 – Interpret an Ethernet Port Report — 4 Marks

Bloom's Level: K5 – Evaluate

A shop-floor PC is slow. Its network interface reports:

```
Speed: 100Mb/s   Duplex: Half   Auto-negotiation: off
Link detected: yes   rx_errors: 2381   late_collisions: 412   crc_errors: 1907
```

Work out:

- What a late collision and a CRC error each mean, and what the combination suggests about the two ends of the link
- The likely cause (a duplex mismatch with a gigabit switch port set to auto) and the exact configuration change on Linux (ethtool) or Windows
- Whether the MAC address 01:00:5e:0a:01:02 belongs to a single device or a group, and what the first three bytes of 3c:52:82:4e:11:07 identify
- Whether jumbo frames (9,000 bytes) would help the CCTV storage server, and what must also change for them to work

Alternative Concept Analysis

Compare a shop floor wired as a ring of industrial switches with fast ring recovery against a star of switches. For one cable cut, state the recovery time each needs and which devices lose connection.

Evidence Expected

A reasoned interpretation of the report, the corrective configuration and correct answers on addressing and frame size.

TASK 7 – Will the Receiver Notice the Error? 4 Marks

Bloom's Level: K5 – Evaluate

Frames between two shop-floor switches are protected by a CRC with generator 1011 ($x^3 + x + 1$).

Work out:

- The CRC bits for the data 1101011011, by hand (show the division)
- Whether a received frame 1101011011101 is accepted or rejected
- The longest burst error this generator is certain to detect, and an error pattern it cannot detect
- The two-dimensional parity of a 4 × 4 block of your choice, and a 4-bit error pattern that two-dimensional parity misses
- The Internet checksum of the four 16-bit words 4500, 0073, 0000, 4000 (hexadecimal)

Complete the Code

Complete TODO 1 below to finish the modulo-2 division, and run the program with Python 3.

```
"""CRC error detection for frames sent between two switches."""



def mod2_div(dividend, divisor):
    """Return the remainder of modulo-2 (XOR) division of two bit strings."""
    rem = list(dividend)
    for i in range(len(dividend) - len(divisor) + 1):
        # TODO 1: If the current leading bit rem[i] is 1, XOR the divisor into rem
        #   starting at position i
        pass  # replace this line with your code
    return "".join(rem[-(len(divisor) - 1):])



def encode(data, gen):
    crc = mod2_div(data + "0" * (len(gen) - 1), gen)
    return data + crc



def check(frame, gen):
    return set(mod2_div(frame, gen)) == {"0"}



GEN = "1011"                                    # x^3 + x + 1
frame = encode("1101011011", GEN)
print("transmitted:", frame)
print("no error   :", check(frame, GEN))
bad = frame[:4] + ("1" if frame[4] == "0" else "0") + frame[5:]
print("1-bit error:", check(bad, GEN))
burst = frame[:2] + "".join("1" if b == "0" else "0" for b in frame[2:5]) + frame[5:]
print("3-bit burst:", check(burst, GEN))
```

Expected output:

```
transmitted: 1101011011100
no error   : True
1-bit error: False
3-bit burst: False
```

Find, with the program, an error pattern that generator 1011 cannot detect, and explain why.

Alternative Concept Analysis

Which errors would a simple even-parity bit catch that the CRC-3 generator 1011 misses, and which would it miss? Give one 2-bit and one 4-bit error pattern for each method and state the result.

Evidence Expected

Hand calculations with working, an undetected-error example for each method and a comparison of their strength.

TASK 8 – Correct the Error or Ask Again? 3 Marks

Bloom's Level: K5 – Evaluate

A vibration sensor sends 16-bit readings over a noisy radio link where retransmission takes 40 ms.

Work out:

- The number of Hamming check bits needed for a 16-bit word (use $2^r \geq m + r + 1$) and the overhead percentage
- The minimum Hamming distance of the code {00000, 01011, 10101, 11110}, and how many errors it can detect and correct
- The corrected data when the Hamming(7,4) codeword 0110011 arrives as 0110111
- Whether forward error correction or retransmission (ARQ) suits this link, and why video streaming and 5G use FEC

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Hamming(7,4) code: correct a single-bit error in a 4-bit sensor reading."""



def encode(d):                      # d = [d1, d2, d3, d4]
    p1 = d[0] ^ d[1] ^ d[3]
    p2 = d[0] ^ d[2] ^ d[3]
    p3 = d[1] ^ d[2] ^ d[3]
    return [p1, p2, d[0], p3, d[1], d[2], d[3]]      # positions 1..7



def decode(c):
    # TODO 1: Compute the syndrome bits s1 (positions 1,3,5,7), s2 (2,3,6,7) and s3
    #   (4,5,6,7); pos = s1 + 2*s2 + 4*s3. If pos is not 0, flip bit pos (index pos-1).
    #   Return pos and the data bits at positions 3, 5, 6, 7
    pass  # replace this line with your code



word = encode([1, 0, 1, 1])
print("codeword       :", "".join(map(str, word)))
for flip in (None, 5):
    rx = word[:]
    if flip:
        rx[flip - 1] ^= 1
    pos, data = decode(rx)
    print(f"error at {str(flip):<4}: syndrome {pos}, data {''.join(map(str, data))}")
```

Expected output:

```
codeword       : 0110011
error at None: syndrome 0, data 1011
error at 5   : syndrome 5, data 1011
```

Flip two bits of the codeword and show that the decoder now "corrects" the wrong bit.

Alternative Concept Analysis

At a bit-error rate of 1 in 1,000, compare sending 16-bit readings with Hamming correction against retransmitting on error: calculate the bits sent per reading and the average delay for each, using the 40 ms retransmission time.

Evidence Expected

Correct calculations, a corrected codeword and a justified choice between FEC and ARQ.

TASK 9 – How Big Must the Window Be? 4 Marks

Bloom's Level: K5 – Evaluate

The head-office link is 400 km of fibre at 1 Gbps; frames are 1,500 bytes; light travels at 2×10^8 m/s in fibre.

Work out:

- The transmission time, the propagation time and the ratio a = propagation / transmission
- The link utilisation with stop-and-wait ARQ – and what it means in Mbps
- The smallest window that keeps the link fully used, and the number of sequence-number bits needed for Go-Back-N and for Selective Repeat with that window
- Which of the two ARQ protocols resends fewer frames after one loss, and what it costs the receiver
- How HDLC or PPP frames mark the start and end of a frame

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Link utilisation of stop-and-wait and sliding-window protocols on different links."""


def utilisation(frame_bits, rate_bps, distance_m, window, speed=2e8):
    tx = frame_bits / rate_bps                  # transmission time
    prop = distance_m / speed                   # propagation time
    # TODO 1: a = prop / tx. Utilisation is window / (1 + 2a), but never more than 1.
    #   Return the utilisation and a
    pass  # replace this line with your code


links = [("Campus LAN, 200 m, 1 Gbps", 1e9, 200),
         ("City fibre, 50 km, 1 Gbps", 1e9, 50_000),
         ("Satellite, 36,000 km, 10 Mbps", 10e6, 36_000_000)]
for name, rate, dist in links:
    u1, a = utilisation(12_000, rate, dist, 1)
    u7, _ = utilisation(12_000, rate, dist, 7)
    u127, _ = utilisation(12_000, rate, dist, 127)
    print(f"{name:<31} a = {a:7.2f}  stop-and-wait {u1:6.1%}  "
          f"W=7 {u7:6.1%}  W=127 {u127:6.1%}")
```

Expected output:

```
Campus LAN, 200 m, 1 Gbps       a =    0.08  stop-and-wait  85.7%  W=7 100.0%  W=127 100.0%
City fibre, 50 km, 1 Gbps       a =   20.83  stop-and-wait   2.3%  W=7  16.4%  W=127 100.0%
Satellite, 36,000 km, 10 Mbps   a =  150.00  stop-and-wait   0.3%  W=7   2.3%  W=127  42.2%
```

Calculate the smallest window that gives 100% utilisation on the satellite link.

Alternative Concept Analysis

Suppose the head-office link used Selective Repeat with a window of 64 instead of the window you calculated. Calculate the resulting utilisation and throughput, and the receiver buffer space needed.

Evidence Expected

Correct timing, utilisation and window calculations, sequence-number sizes and a justified protocol choice.

TASK 10 – Find Where the Delay Comes From — 4 Marks

Bloom's Level: K5 – Evaluate

A traceroute from the plant to the head-office server gives:

```
 1  10.10.1.1        0.6 ms   0.5 ms   0.6 ms
 2  100.64.0.1       3.1 ms   2.9 ms   3.4 ms
 3  172.31.4.9       4.0 ms  38.2 ms   4.2 ms
 4  103.21.8.17     11.8 ms  11.5 ms  12.1 ms
 5  * * *
 6  10.200.0.10     12.6 ms  12.4 ms  12.9 ms
```

Work out:

- The propagation delay you would expect for 400 km of fibre, and how much of the measured delay it explains
- Why hop 3 shows one value of 38.2 ms, and why hop 5 shows only stars – are they faults?
- Which hop adds the most delay
- Your own traceroute, ping and Wireshark capture of a real connection, with the headers of each layer identified

Alternative Concept Analysis

Compare continuous monitoring with SNMP counters and flow records (NetFlow) against occasional ping and traceroute tests: which of the problems in the traceroute above would each have detected, and how quickly?

Evidence Expected

A reasoned reading of the traceroute, a propagation-delay check and your own measured evidence.

TASK 11 – Defend the Network Design — 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the network design for the plant.

Work out:

- The medium, wireless technology and Ethernet settings for each area
- The error-control and flow-control method for each link, with the calculation that justifies it
- The two biggest risks that remain in your design
- Whether your design still works for five plants

Your recommendation must quote **numbers from Tasks 1–10**.

Alternative Concept Analysis

Suppose the whole plant used private 5G instead of a wired backbone with Wi-Fi. Using your traffic figures from Task 1, decide which traffic 5G should carry and which must stay on cable, and justify the split.

Evidence Expected

A recommendation supported by your own calculations and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design — 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The company adds a **private 5G network** on the shop floor and **50 AGVs** that move parts between lines and must never miss a command.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which media, wireless choices and error-control settings are affected, and recalculate the traffic from Task 1 with the AGVs added.

Evidence Expected

A quantified analysis of how the changes affect the original design.

B. Test the Original Design – 2 Marks

Show, using your hand-over and delay calculations, whether the original design can serve 50 moving AGVs.

Evidence Expected

A calculation-based test of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised design (for example: 5G URLLC for AGVs and robots, Wi-Fi 6 for offices, fibre backbone).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision with numbers and the trade-offs involved.

Evidence Expected

A well-supported final decision.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Traffic Sizing	K5 – Evaluate	4
Task 2 – Collision and Broadcast Domains, Standards	K5 – Evaluate	3
Task 3 – Reading a Frame Layer by Layer	K5 – Evaluate	4
Task 4 – Transmission Media	K5 – Evaluate	4
Task 5 – Wi-Fi 6 or 5G	K5 – Evaluate	4
Task 6 – Ethernet Port Report	K5 – Evaluate	4
Task 7 – Error Detection	K5 – Evaluate	4
Task 8 – Error Correction	K5 – Evaluate	3
Task 9 – Window Size and ARQ	K5 – Evaluate	4
Task 10 – Delay Analysis	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, calculations, Wireshark captures, measurements and evaluations.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

COMPUTER NETWORKS (CSA07) – ASSIGNMENT 1 (CO1)

Make the Data of a Smart Manufacturing Plant Arrive Correctly and On Time – Answer Sheet

Name	AB Sai Saranya	Register No.	1921260020
Course	CSA07 – Computer Networks	Total marks	50
GitHub repository	CSA07-1921260020-Assignment1	Tools used	Python 3, Cisco Packet Tracer, Wireshark

Scenario. Automobile-parts smart plant: robots need commands within 10 ms; hundreds of sensors; CCTV streams; office ERP/e-mail; production data exchanged with a head office 400 km away. The contractor proposes ordinary office Wi-Fi everywhere and one broadband link. Engineers fear welding noise, delayed robot commands and a slow head-office link. Growth: 1 line (40 devices) → 1 plant (1,500) → 5 plants.

Design Challenge (Task 12): private 5G on the shop floor and 50 AGVs – re-evaluate the design (retain / modify / redesign).

TASK 1 – Size the Traffic Before Choosing Anything (4 marks)

QUESTION

One line: Robot controllers 40 × 200-byte command every 2 ms; Vibration sensors 600 × 50 B/s; CCTV 24 × 4 Mbps continuous; Office users 150 × 2 Mbps at peak, 10% active. Work out: (a) total traffic of one line and of 8 lines; (b) which source sends most data and which has the strictest delay – same?; (c) is a 1 Gbps uplink enough, and what happens to robot commands if CCTV shares the queue; (d) rank traffic by harm of a 100 ms delay. **Alternative:** CCTV records locally and sends motion clips (~5% of the time) – recalculate; is 1 Gbps still the bottleneck?

(a) Traffic per source

Source	Calculation	Traffic
Robot controllers	40 × 200 B × 8 bit ÷ 0.002 s = 40 × 0.8 Mbps	32.00 Mbps
Vibration sensors	600 × 50 B × 8 bit = 240,000 bit/s	0.24 Mbps
CCTV cameras	24 × 4 Mbps	96.00 Mbps
Office users	150 × 10% × 2 Mbps = 15 active × 2 Mbps	30.00 Mbps
One line total	32 + 0.24 + 96 + 30	**158.24 Mbps**
Plant (8 lines)	8 × 158.24	**1,265.92 Mbps ≈ 1.27 Gbps**

(b), (c) Largest vs strictest, and the 1 Gbps uplink

- **Most data:** CCTV (96 of 158.24 Mbps = 60.7%). **Strictest delay:** robot commands (10 ms deadline) although they are only 20% of the volume (32 Mbps). They are **not the same** – volume and urgency are different properties, so a design sized only for bandwidth will still fail the robots.
- **Uplink:** 1,265.92 Mbps > 1,000 Mbps, so one 1 Gbps uplink is **not enough** for 8 lines (126.6% load, ≈ 266 Mbps short). One line alone (158 Mbps = 15.8%) fits easily.
- **Shared queue:** a FIFO queue carries a 1,500-byte CCTV packet which takes 12 µs per packet on 1 Gbps; but when the queue is over-subscribed, bursts of camera packets build a queue of milliseconds (e.g. 100 packets ahead = 1.2 ms; a 1 MB switch buffer full = 8 ms at 1 Gbps) and robot commands wait behind them and may be dropped – robot commands then miss the 10 ms deadline and the jitter is unpredictable. Hence robot traffic needs its own VLAN/queue with strict priority (802.1p / DSCP EF) or physically separate links.

(d) Ranking by harm of a 100 ms delay

Rank	Traffic	Reason
1 (worst)	Robot commands	Deadline 10 ms – 100 ms is 10× too late: ten control cycles missed, safety stop or collision/scrap.
2	Office voice / IP-phone & ERP interactive	Voice is unusable above ≈150 ms one-way; 100 ms is the edge of acceptable, ERP feels slow but works.
3	CCTV video	A playout buffer hides 100 ms; only live PTZ control is affected.
4	Vibration sensors	50 B/s samples, trend data analysed over seconds; a late sample is harmless.
5 (least)	E-mail / bulk file	Elastic traffic, tolerates seconds.

Alternative concept – motion-triggered CCTV (5% duty)

Source	Traffic
Robots	32 Mbps
Vibration	0.24 Mbps
CCTV 96 × 0.05	4.80 Mbps
Office	30 Mbps

Source	Traffic
One line	**67.04 Mbps**
8 lines	**536.32 Mbps (53.6% of 1 Gbps)**

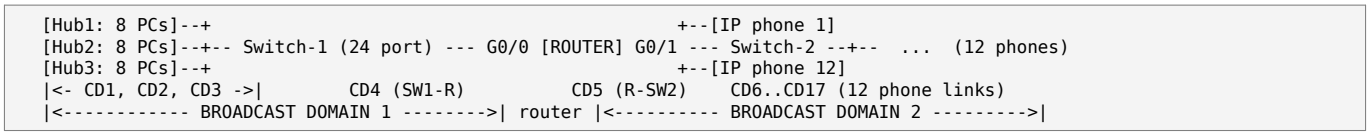
Decision: traffic falls by 57.6% (158.24 → 67.04 Mbps per line). The 1 Gbps uplink is **no longer the bottleneck** (53.6% utilisation, below the usual 70% planning limit). The bottleneck moves to delay (queueing) rather than bandwidth, so QoS is still needed. Trade-off: local storage cost and clips missed if motion detection fails.

Evidence to attach: Excel/Python calculation sheet and a delay-versus-volume chart (Task01 folder).

TASK 2 – Count the Domains in the Contractor's Drawing (3 marks)

QUESTION
Drawing: three hubs with 8 PCs each, all three hubs connected to one 24-port switch; the switch to router G0/0; router G0/1 to a second switch with 12 IP phones. Work out: collision and broadcast domains with marked drawing; OSI layer of each device and why a hub cannot stop collisions; point-to-point vs multipoint connections; the standard and body for Ethernet 802.3, Wi-Fi 802.11, IPv4 RFC 791, OSI ISO 7498, 5G – and why only standard-compliant equipment.
Alternative: replace the three hubs with unmanaged switches – recount.

Marked drawing (CD = collision domain, BD = broadcast domain):



Item	Count	Reasoning
Collision domains	**17**	Each hub + its 8 PCs + its switch port is ONE domain (3). Switch-1↔router G0/0 (1), router G0/1↔Switch-2 (1), and each of the 12 phone-to-switch ports (12): 3+1+1+12 = 17.
Broadcast domains	**2**	Only the router (layer 3) separates broadcast domains: the G0/0 side (24 PCs + switch) and the G0/1 side (12 phones).

Device	OSI layer	Behaviour
Hub	Layer 1 (Physical)	Repeats bits on all ports; cannot read frames, so it cannot isolate collisions – all 8 PCs (and anything behind the hub) share one wire and CSMA/CD.
Switch	Layer 2 (Data link)	Learns MAC addresses, forwards per port; each port is its own collision domain; broadcasts are still flooded.
Router	Layer 3 (Network)	Routes by IP; blocks broadcasts; one broadcast domain per interface.
IP phone / PC	Layers 1–7	End hosts.

Point-to-point: switch–router G0/0, router G0/1–switch-2, switch-1–each hub uplink, every phone–switch cable, (PC–hub cables are to a shared repeater). **Multipoint / shared:** each hub segment (8 PCs + uplink share one medium), i.e. the hub-based CD is multipoint.

Technology	Standard	Body	Why
Ethernet	IEEE 802.3	IEEE	MAC/PHY framing, speeds, cabling
Wi-Fi	IEEE 802.11	IEEE	WLAN MAC/PHY, Wi-Fi Alliance certifies
IPv4	RFC 791	IETF	Addressing and datagram format
OSI model	ISO 7498	ISO	Reference layering
5G	3GPP Rel-15/16 specs; IMT-2020	3GPP / ITU-R	Radio, core, URLLC

Why standard-compliant equipment: interoperability between vendors, no lock-in, predictable behaviour (e.g. 90 m limits, frame formats), safety/EMC compliance, and future upgrades (5G, Wi-Fi 6) without replacing the plant.

Alternative – three unmanaged switches instead of hubs

	Hubs (original)	Switches (proposal)
Collision domains	17	24 PC links + 3 uplinks + 1 + 1 + 12 = **41**
Broadcast domains	2	**2** (unchanged)

Fixes: collisions (full duplex, 100 Mbps per PC instead of 100 Mbps shared by 8), throughput and latency. **Does not fix:** broadcast domain size, no VLAN separation of robots/CCTV/office, no security or traffic prioritisation (unmanaged), no loop protection.

Evidence to attach: Packet Tracer screenshot of the drawing with domains marked.

TASK 3 – Read a Captured Frame Layer by Layer (4 marks)

QUESTION
Capture of one robot command: Ethernet II Dst 00:1d:9c:c7:b0:41 Src 3c:52:82:4e:11:07 Type 0x0800; IPv4 Src 10.10.1.25 Dst 10.10.9.4 TTL 61 Protocol 6 (TCP); TCP Src 51522 Dst 502 (Modbus) Seq 1201 Ack 88 Len 12; Data 12 bytes. Work out: OSI layer and PDU of each line; routers crossed if TTL started at 64; bytes on the wire (+4-byte FCS) and % overhead, repeated for a 1,400-byte camera packet; which layer is responsible when (a) MAC wrong, (b) TTL 0, (c) port closed; TCP/IP to OSI mapping. **Complete the Code** (TODO 1). **Alternative:** EtherType 0x8892 with no IP/TCP.

Line	OSI layer	PDU	Key fields
Ethernet II	2 – Data link	Frame	MAC src/dst, EtherType 0x0800 → IPv4
IPv4	3 – Network	Packet	IP addresses, TTL 61, protocol 6 = TCP
TCP	4 – Transport	Segment	Ports 51522 → 502, Seq/Ack, Len 12
Data (Modbus)	7 – Application	Data / message	Write-register command, 12 bytes

- **Routers crossed:** 64 − 61 = **3 routers** (each router decrements TTL by 1).
- **Robot command bytes:** 14 (Ethernet) + 20 (IP) + 20 (TCP) + 12 (data) + 4 (FCS) = **70 bytes**. Overhead = 58 ÷ 70 = **82.9%**.
- **Camera packet:** 14 + 20 + 20 + 1400 + 4 = **1,458 bytes**; overhead = 58 ÷ 1458 = **3.98%**.
- **Meaning:** the header cost is fixed (58 B), so tiny control messages are ≈ 83% header while video is ≈ 4%. Small robot commands are therefore very 'expensive' per useful byte – they need low-latency priority more than they need bandwidth, and protocol choice (lighter headers) matters.
- **Fault → layer:** (a) wrong MAC → Layer 2 (frame not accepted by NIC / dropped by switch); (b) TTL reaches 0 → Layer 3 (router drops, sends ICMP Time Exceeded); (c) closed destination port → Layer 4 (host replies TCP RST / ICMP port unreachable).
- **TCP/IP mapping:** Application + Presentation + Session (OSI 7-6-5) → Application; Transport (4) → Transport; Network (3) → Internet; Data link + Physical (2-1) → Network access (Link).

Completed program (TODOs replaced; file: t3.py)

```
"""How data is wrapped by each layer at the sender and unwrapped at the receiver."""
LAYERS = [("Application", "HTTP"), ("Transport", "TCP"), ("Network", "IP"),
          ("Data link", "ETH")]

def send(message):
    pdu = message
    for layer, tag in LAYERS:
        head = "[" + tag + "|"
        tail = "|FCS]" if tag == "ETH" else "]"
        pdu = head + pdu + tail
        print(f"{layer:<12} adds {tag:<4} -> {len(pdu):3d} bytes")
    return pdu

def receive(frame):
    pdu = frame
    for layer, tag in reversed(LAYERS):
        head = "[" + tag + "|"
        tail = "|FCS]" if tag == "ETH" else "]"
        assert pdu.startswith(head) and pdu.endswith(tail), f"{layer}: bad header"
        pdu = pdu[len(head):-len(tail)]
    return pdu

frame = send("GET /results")
print("on the wire:", frame)
print("delivered  :", receive(frame))
print(f"overhead   : {len(frame) - len('GET /results')} bytes for 12 bytes of data")
```

Output obtained (run with Python 3):

```
Application  adds HTTP ->  19 bytes
Transport    adds TCP  ->  25 bytes
Network      adds IP   ->  30 bytes
Data link    adds ETH  ->  40 bytes
on the wire: [ETH|[IP|[TCP|[HTTP|GET /results]]]|FCS]
delivered  : GET /results
overhead   : 28 bytes for 12 bytes of data
```

1,400-byte message: the toy model always adds 28 bytes. For 12 bytes of data the frame is 40 bytes, so overhead = 28 ÷ 40 = 70%; for 1,400 bytes the frame is 1,428 bytes, so overhead = 28 ÷ 1,428 = 1.96%. The same headers cost 70% on a small command but only 2% on a big message – small commands are inefficient, which agrees with the 82.9% vs 3.98% result for the real capture.

Alternative – command directly inside an Ethernet frame (EtherType 0x8892)

	With IP + TCP	Raw Ethernet 0x8892
Bytes on wire	14+20+20+12+4 = 70	14+12+4 = 30 (padded to 64 minimum)
Commands/s on 100 Mbps (nominal)	100e6 ÷ 560 = 178,571	100e6 ÷ 240 = 416,667
With 64-byte minimum frame	–	100e6 ÷ 512 = 195,313

Bytes saved per command: 40 (IP 20 + TCP 20) – 57% less. Nominally 238,095 more commands/s; because Ethernet pads to 64 bytes the real gain is 6 bytes (70 → 64), about 17,700 more commands/s. **Lost by giving up IP:** routing between subnets (command can never leave its LAN), TTL, reliable delivery/retransmission and flow control of TCP, IP-level firewalling, VPN to the head office, standard diagnostics (ping, traceroute). Acceptable only inside one cell/VLAN (as PROFINET-RT / EtherCAT do).

Evidence to attach: Wireshark capture screenshot with each layer expanded; GitHub folder Task03.

TASK 4 – Choose a Medium for Each Run (4 marks)

QUESTION
Runs: (a) robot cell next to arc-welders 85 m; (b) office floor to shop-floor switch 140 m; (c) shop floor to data centre 550 m; (d) plant to head office 400 km. Work out: a medium per run; which run cannot use copper at all; Shannon capacity of a 250 MHz cable at SNR 40 dB and 15 dB; why measured speed is below Shannon. **Complete the Code** (TODO 1, 2); find the SNR for 56 kbps on a 3,100 Hz line. **Alternative:** Wi-Fi 6, 160 MHz, SNR 15 dB for the robot cell.

Run	Distance	Medium chosen	Deciding reason
(a) Robot cell by welders	85 m	**Cat6A S/FTP** (shielded), or multimode fibre if EMI is severe	Within 90 m permanent link, but welding produces strong EMI – shielding with proper grounding; fibre is immune.
(b) Office → shop-floor switch	140 m	**Multimode fibre** (OM3/OM4) – the trap	140 m > 100 m channel / 90 m permanent link of copper; Cat6 would give errors/loss of link.
(c) Shop floor → data centre	550 m	**Single-mode fibre (OS2)**	Multimode gives only 300–400 m at 10G; 550 m is at the 1G multimode edge; SM is future-proof for 10/40G.
(d) Plant → head office	400 km	**Leased line / dark fibre (single-mode) or MPLS from a carrier**	Any LAN medium is impossible at 400 km; needs repeaters/amplifiers owned by a carrier. Wireless unsuitable for 1 Gbps reliability.

Which run cannot use copper Ethernet at all? Run (d): 400 km is 4,000 times the 100 m copper limit (attenuation); runs (b) 140 m and (c) 550 m also exceed 100 m, but a switch/repeater could be added every 100 m; for (d) copper is impossible in practice. The 'trap' is (b): it looks like an ordinary office cable, but 140 m violates the TIA/EIA-568 limit.

Shannon capacity $C = B \cdot \log_2(1+SNR)$:

Case	SNR ratio	C = 250 MHz × $\log_2(1+SNR)$
40 dB (office cable)	10,000	250e6 × 13.288 = **3,321.96 Mbps**
15 dB (near welders)	31.62	250e6 × 5.028 = **1,256.95 Mbps**

Noise cuts the theoretical capacity by 62% (3,322 → 1,257 Mbps). Still above 1 Gbps, but with almost no margin – bit errors, retransmissions and latency jitter would rise sharply once real-world losses (below) are included. The robot cell must use shielded cable or fibre to keep the SNR high. **Measured speed is always below Shannon** because real systems use finite coding/modulation, protocol overhead (preamble, headers, inter-frame gap), hardware imperfections, interference/crosstalk and the margin needed for an acceptable error rate.

Completed program (TODOs replaced; file: t4.py)

```
"""Nyquist and Shannon limits for the transmission media proposed for a campus."""
import math

def nyquist(bandwidth_hz, levels):
    return 2 * bandwidth_hz * math.log2(levels)

def shannon(bandwidth_hz, snr_db):
    snr = 10 ** (snr_db / 10)
    return bandwidth_hz * math.log2(1 + snr)

media = [("Telephone line", 3_100, 30), ("Cat6 pair", 250e6, 40),
         ("Wi-Fi 6 channel", 160e6, 25)]
for name, bw, snr in media:
    print(f"{name:<16} Shannon limit {shannon(bw, snr) / 1e6:10.2f} Mbps")
print(f"Nyquist, 3,100 Hz with 4 levels: {nyquist(3_100, 4) / 1e3:.1f} kbps")
# extra: SNR needed for 56 kbps on 3,100 Hz
need = 2 ** (56_000 / 3_100) - 1
print(f"SNR for 56 kbps on 3,100 Hz: {10 * math.log10(need):.1f} dB")
```

Output obtained (run with Python 3):

```
Telephone line   Shannon limit       0.03 Mbps
Cat6 pair        Shannon limit    3321.96 Mbps
Wi-Fi 6 channel  Shannon limit    1329.50 Mbps
Nyquist, 3,100 Hz with 4 levels: 12.4 kbps
SNR for 56 kbps on 3,100 Hz: 54.4 dB
```

56 kbps on 3,100 Hz: 56,000 ÷ 3,100 = 18.06 bit/Hz → SNR = $2^{18.06} - 1 \approx 2.7 \times 10^5$ = **54.4 dB**. A voice line has about 30–35 dB, so it is **not realistic** for an analog line (56k modems only reached it because the downstream path was digital with no quantisation noise).

Alternative – Wi-Fi 6 for the robot cell

C = 160 MHz × $\log_2(1 + 31.62)$ = 160e6 × 5.028 = **804.4 Mbps**. Capacity is plenty, but **not acceptable for 10 ms robot commands**: Shannon limits capacity, not latency. CSMA/CA contention, retransmissions, interference from welding arcs, roaming and unpredictable queueing make a hard 10 ms guarantee impossible; use shielded cable/fibre for the fixed robots (and private 5G URLLC for mobile equipment).

Evidence to attach: Output of python t4.py (screenshot) and cable-selection table.

TASK 5 – Wi-Fi 6 or 5G for Moving Equipment? (4 marks)

QUESTION
AGV speed 2 m/s over a 1.2 km route. Wi-Fi APs 30 m apart, hand-over 150 ms; private 5G cell 300 m, hand-over 20 ms; AGV commands have a 10 ms deadline. Work out: hand-overs per trip and unreachable time for Wi-Fi and 5G; which 5G service type (eMBB, URLLC, mMTC) the AGVs, cameras and sensors need; where Bluetooth/Zigbee/LoRa is better; whether Wi-Fi 6 with 802.11r meets the deadline. **Alternative:** 600 sensors × 50 B/s on one LoRa gateway of ~5 kbps.

	Wi-Fi 6 (30 m APs)	Private 5G (300 m cells)
Trip time	1,200 m ÷ 2 m/s = 600 s	600 s
Cells along route	1,200 ÷ 30 = 40	1,200 ÷ 300 = 4
Hand-overs per trip	40 cells → **39** (≈ 40 with start)	4 cells → **3**
Unreachable time	39 × 150 ms = **5.85 s**	3 × 20 ms = **60 ms**
10 ms commands missed per trip	5.85 s ÷ 10 ms = **585**	60 ms ÷ 10 ms = **6**
Hand-over every	15 s (every 30 m ÷ 2 m/s)	150 s

Even 5G's 20 ms hand-over is longer than the 10 ms deadline, so each 5G hand-over loses 2 commands; with dual connectivity / conditional hand-over (≈0 ms) and an AGV that holds its last safe command, the loss is removed. Wi-Fi, losing 15 consecutive commands every 15 s, is not acceptable.

Traffic	5G service	Why
AGV control	**URLLC**	1–10 ms latency, 99.999% reliability
CCTV cameras	**eMBB**	High bandwidth (4 Mbps × 24)
Vibration/temperature sensors	**mMTC**	Massive number of low-rate, low-power devices

- **Bluetooth (BLE):** operator handhelds, tool tags, PPE/worker-presence beacons within ≈10 m. **Zigbee:** mesh of low-rate sensors and lighting/actuators in a cell, low power. **LoRa/LoRaWAN:** yard, warehouse or outdoor slow sensors (tank levels, energy meters) over kilometres on batteries – not for robot control or video.
- **Wi-Fi 6 with 802.11r:** fast roaming reduces hand-over to roughly 20–50 ms, which is still greater than 10 ms, so it **cannot guarantee** the deadline. Remaining risks: contention (CSMA/CA), interference from welders and neighbouring cells, retries and unpredictable queueing, client-driven roaming decisions.

Alternative – LoRaWAN for the sensors

Demand = 600 × 50 B × 8 = **240 kbps** versus the gateway capacity of ~5 kbps: **48 times over capacity** (one gateway supports only 5,000 ÷ 400 = 12.5 sensors at 50 B/s; LoRa duty-cycle limits make it worse). So the 50 B/s vibration sensors **stay on Wi-Fi/5G (mMTC)**; only slow sensors (e.g. temperature, one 50-byte reading per minute = 6.7 bps each → 600 sensors = 4 kbps, fits if ALOHA efficiency and duty cycle allow, practically ~2 gateways) should move to LoRaWAN.

Evidence to attach: Calculation sheet; mapping table; (optional) Packet Tracer wireless topology.

TASK 6 – Interpret an Ethernet Port Report (4 marks)

QUESTION
PC interface: Speed 100 Mb/s, Duplex Half, Auto-negotiation off, Link yes, rx_errors 2381, late_collisions 412, crc_errors 1907. Work out: meaning of a late collision and a CRC error and what the combination says; likely cause and exact fix (ethtool / Windows); is MAC 01:00:5e:0a:01:02 single or group and what do the first 3 bytes of 3c:52:82:4e:11:07 identify; would 9,000-byte jumbo frames help the CCTV storage server and what else must change. **Alternative:** ring of industrial switches with fast recovery vs star.

- **Late collision:** a collision detected after the first 512 bit-times (64 bytes) – on a correct half-duplex network all collisions occur earlier. It happens when the other end is transmitting full duplex (does not defer) or the cable/segment is too long. **CRC error:** the frame arrived but its FCS does not match – the frame was corrupted or truncated (by the collision).
- **Combination:** the two ends disagree on duplex. The PC end is forced to 100 Mb/s half with auto-negotiation off, while the switch port (auto) assumes full duplex and sends whenever it wants – so the PC sees late collisions and truncated frames with bad CRCs (412 late collisions + 1,907 CRC errors ≈ 97% of the 2,381 rx_errors).

- **Cause:** duplex mismatch (manual setting on one side vs auto on the other). **Fix (Linux):** `sudo ethtool -s eth0 autoneg on` (or force both ends: `sudo ethtool -s eth0 speed 1000 duplex full autoneg off` and set the same on the switch) and verify with `ethtool eth0`. **Windows:** Device Manager → adapter → Properties → Advanced → Speed & Duplex → Auto Negotiation. Best practice: auto on both ends (mandatory for 1000BASE-T).
- **MAC 01:00:5e:0a:01:02:** the least-significant bit of the first byte is 1 (0x01), so it is a **group (multicast)** address – the 01:00:5e prefix is the IPv4 multicast mapping (group x.10.1.2). **3c:52:82** is the OUI – the 24-bit IEEE-assigned manufacturer (vendor) identifier of the NIC (look it up in the IEEE OUI registry); the last 3 bytes 4e:11:07 identify the individual card.
- **Jumbo frames:** with 9,000-byte payload header overhead drops from 58 ÷ 1,518 = 3.8% to 58 ÷ 9,058 = 0.64%, with ~6× fewer packets and interrupts, so it **helps bulk CCTV storage traffic** (higher throughput, lower CPU). Everything on the path must change too: the MTU on the server NIC, the storage, every switch port and VLAN in the path (otherwise frames are dropped or fragmented); not on the robot VLAN, where large frames add delay (9,000 B takes 72 µs at 1 Gbps vs 12 µs).

Alternative – ring of industrial switches vs star

	Ring with fast recovery (MRP/RSTP)	Star of switches
One cable cut between two switches	Ring reopens the blocked link; recovery ≈ 10–200 ms (MRP ≤ 200 ms; RSTP ≈ 1–2 s); **no device loses connection** permanently, but traffic is lost during recovery	Cable to one access switch: **that switch and all devices behind it lose connection** until repair (unless a redundant uplink exists); cut to a single device only isolates that device
Recovery time	10–200 ms (≥ 20 missed 10 ms commands at 200 ms); zero with PRP/HSR	None automatic (no alternate path) – manual repair

Decision: ring (or dual-star/PRP) for the shop-floor backbone to survive one cut; star for access ports; robot cells that cannot tolerate 200 ms need PRP/HSR (0 ms).

Evidence to attach: Output of ethtool/ipconfig before and after the fix (screenshots).

TASK 7 – Will the Receiver Notice the Error? (4 marks)

QUESTION
Generator 1011 (x^3+x+1). Work out: CRC bits for data 1101011011 by hand; whether received frame 1101011011101 is accepted; the longest burst certain to be detected and an undetectable error; 2D parity of a 4×4 block and a 4-bit pattern that it misses; Internet checksum of 4500, 0073, 0000, 4000 (hex). **Complete the Code** (TODO 1) and find an undetectable pattern. **Alternative:** what even parity catches that CRC-3 misses and vice versa.

CRC by hand (modulo-2 division)

Data 1101011011 + 3 zeros (degree of generator) → dividend 1101011011000, divisor 1011. Each XOR step:

```
1101011011000
1011   (XOR at bit 1)
 1011   (XOR at bit 2)
  1011   (XOR at bit 3)
   1011   (XOR at bit 4)
     1011   (XOR at bit 6)
       1011   (XOR at bit 8)
Remainder = 100   →   CRC = 100   →   transmitted frame = 1101011011100
```

- **CRC = 100**; transmitted frame 1101011011100 (matches the program output).
- **Received 1101011011101:** dividing gives remainder 001 ≠ 000, so the frame is **rejected** (the last bit was flipped).
- **Burst length:** the generator has degree 3 (3 check bits), so every burst of length ≤ 3 bits is detected with certainty (a 4-bit burst is missed with probability 2^{-2} = 1/4 and longer ones with 1/8). **Undetectable pattern:** any error pattern that is a multiple of 1011, e.g. 0000000001011 (the generator itself) or two bit errors 7 apart (x^7+1 is divisible by x^3+x+1).
- **Internet checksum:** 4500 + 0073 + 0000 + 4000 = 8573 (no carry); one's complement = FFFF − 8573 = **7A8C**.

Two-dimensional parity (even) of a 4 × 4 block

	c1	c2	c3	c4	Row parity
r1	1	0	1	1	1
r2	0	1	1	0	0
r3	1	1	0	0	0
r4	0	0	1	1	0
Col parity	0	0	1	0	–

Undetected 4-bit pattern: flip the four corners of a rectangle, e.g. r1c1, r1c2, r2c1, r2c2 (1 0 / 0 1 become 0 1 / 1 0) – each affected row and column has two flips, so all row and column parities still match. (Any single, double or triple error is detected; any rectangle of four is missed.)

Completed program (TODOs replaced; file: t7.py)

```
"""CRC error detection for frames sent between two switches."""
def mod2_div(dividend, divisor):
    rem = list(dividend)
    for i in range(len(dividend) - len(divisor) + 1):
        if rem[i] == "1":
            for j in range(len(divisor)):
                rem[i + j] = "0" if rem[i + j] == divisor[j] else "1"
    return "".join(rem[-(len(divisor) - 1):])

def encode(data, gen):
    crc = mod2_div(data + "0" * (len(gen) - 1), gen)
    return data + crc

def check(frame, gen):
    return set(mod2_div(frame, gen)) == {"0"}

GEN = "1011"
frame = encode("1101011011", GEN)
print("transmitted:", frame)
print("no error   :", check(frame, GEN))
bad = frame[:4] + ("1" if frame[4] == "0" else "0") + frame[5:]
print("1-bit error:", check(bad, GEN))
burst = frame[:2] + "".join("1" if b == "0" else "0" for b in frame[2:5]) + frame[5:]
print("3-bit burst:", check(burst, GEN))
# undetectable error: XOR the frame with the generator itself (1011) at the end
und = frame[:-4] + "".join("1" if a != b else "0" for a, b in zip(frame[-4:], GEN))
print("error pattern 1011 :", "accepted (UNDETECTED)" if check(und, GEN) else "detected")
und2 = "".join("1" if (i in (0, 7)) != (c == "1") else "0" for i, c in enumerate(frame))
print("2-bit error, 7 apart:", "accepted (UNDETECTED)" if check(und2, GEN) else "detected")
```

Output obtained (run with Python 3):

```
transmitted: 1101011011100
no error   : True
1-bit error: False
3-bit burst: False
error pattern 1011 : accepted (UNDETECTED)
2-bit error, 7 apart: accepted (UNDETECTED)
```

Why 1011 itself is undetectable: the received frame minus the generator is still divisible by the generator (remainder 000). In general E(x) = G(x)·K(x) cannot be detected.

Alternative – single even-parity bit vs CRC-3 (1011)

Error	Even parity	CRC-3 (1011), 13-bit frame
1-bit	Detected	Detected
2-bit adjacent (e.g. 11)	**Missed** (even number of flips)	Detected
2-bit 7 positions apart	Missed	**Missed** (x^7+1 divisible by G)
3-bit pattern 1011	**Detected** (odd number)	**Missed** (multiple of G)
4-bit pattern 1111	**Missed**	Detected (not a multiple of G)
4-bit pattern 11101 (= 1011 × 11)	Missed	**Missed**
Burst ≤ 3	Odd weights only	**All detected**

Parity catches what CRC misses: odd-weight multiples of 1011, such as the 3-bit 1011. **Parity misses what CRC catches:** every even-weight error that is not a multiple of G (e.g. 11, 1111). CRC is much stronger overall (all bursts ≤ 3, all 1- and 2-bit errors within 7 bits), while parity costs only 1 bit.

Evidence to attach: Python output screenshot; hand-working photo/scan.

TASK 8 – Correct the Error or Ask Again? (3 marks)

QUESTION

A vibration sensor sends 16-bit readings over a noisy radio link where retransmission takes 40 ms. Work out: Hamming check bits for a 16-bit word and overhead %; minimum Hamming distance of {00000, 01011, 10101, 11110} and its detect/correct ability; the corrected data when Hamming(7,4) codeword 0110011 arrives as 0110111; FEC vs ARQ for this link and why video and 5G use FEC. **Complete the Code** (TODO 1); flip two bits and show wrong correction. **Alternative:** BER 1 in 1,000: Hamming vs retransmission.

- **Check bits:** $2^r \geq m + r + 1$ with m = 16: r = 4 gives 16 ≥ 21 (false); r = 5 gives 32 ≥ 22 (true) → **r = 5**. Codeword = 21 bits; overhead = 5 ÷ 16 = **31.25%** of the data (23.8% of the codeword).
- **Minimum distance:** d(00000,01011)=3, d(00000,10101)=3, d(00000,11110)=4, d(01011,10101)=4, d(01011,11110)=3, d(10101,11110)=3 → $\mathbf{d_{min} = 3}$: detects up to d−1 = **2** errors and corrects ⌊(d−1)/2⌋ = **1** error.

- **Received 0110111** (sent 0110011): syndrome s1 (positions 1,3,5,7: 0,1,1,1) = 1; s2 (2,3,6,7: 1,1,1,1) = 0; s3 (4,5,6,7: 0,1,1,1) = 1 → position = 1 + 0 + 4 = **5**. Flip bit 5 → codeword 0110011, data bits (positions 3,5,6,7) = **1011**.
- **FEC or ARQ?** A retransmission costs 40 ms and the link is noisy; FEC corrects at once with a fixed 31% overhead and no feedback channel – best for sensors where every reading is time-critical and the sender is battery-powered (no waiting for ACK). Video and 5G use FEC because retransmitting arrives too late for real-time playback and the radio error rate is high.

Completed program (TODOs replaced; file: t8.py)

```
"""Hamming(7,4) code: correct a single-bit error in a 4-bit sensor reading."""
def encode(d):
    p1 = d[0] ^ d[1] ^ d[3]
    p2 = d[0] ^ d[2] ^ d[3]
    p3 = d[1] ^ d[2] ^ d[3]
    return [p1, p2, d[0], p3, d[1], d[2], d[3]]

def decode(c):
    c = c[:]
    s1 = c[0] ^ c[2] ^ c[4] ^ c[6]
    s2 = c[1] ^ c[2] ^ c[5] ^ c[6]
    s3 = c[3] ^ c[4] ^ c[5] ^ c[6]
    pos = s1 + 2 * s2 + 4 * s3
    if pos:
        c[pos - 1] ^= 1
    return pos, [c[2], c[4], c[5], c[6]]

word = encode([1, 0, 1, 1])
print("codeword       :", "".join(map(str, word)))
for flip in (None, 5):
    rx = word[:]
    if flip:
        rx[flip - 1] ^= 1
    pos, data = decode(rx)
    print(f"error at {str(flip):<4}: syndrome {pos}, data {''.join(map(str, data))}")
# two-bit error: flip positions 3 and 5
rx = word[:]; rx[2] ^= 1; rx[4] ^= 1
pos, data = decode(rx)
print(f"2 errors (3,5): syndrome {pos} -> wrongly flips bit {pos}, data {''.join(map(str, data))} (sent 1011)")
```

Output obtained (run with Python 3):

```
codeword       : 0110011
error at None: syndrome 0, data 1011
error at 5   : syndrome 5, data 1011
2 errors (3,5): syndrome 6 -> wrongly flips bit 6, data 0101 (sent 1011)
```

Two errors: Hamming(7,4) has $d_{min} = 3$, so it can only correct one error; with positions 3 and 5 flipped the syndrome is 6 and the decoder flips the innocent bit 6, producing 0101 instead of 1011 – a silent wrong correction (it must be combined with an overall parity bit/CRC to detect double errors).

Alternative – BER 10^{-3}, 16-bit reading

	Hamming (21,16) FEC	ARQ (retransmit on error)
Bits per reading	21	16 ÷ (1−0.0159) = 16.26 bits average (plus ACK)
P(reading wrong)	P(≥2 errors in 21 bits) = 2.1×10^{-4}	$1 - 0.999^{16} = 1.59\%$
Average extra delay	0.0002 × 40 ms = 0.008 ms	0.0159 ÷ 0.9841 × 40 ms = 0.65 ms (40 ms for 1.6% of readings)
Worst case	40 ms (rare)	40 ms, repeated if the retry fails

Decision: at BER 10^{-3} ARQ sends fewer bits (16.3 vs 21) and its average delay is only 0.65 ms, but 1.6% of readings are late by 40 ms (jitter). If the sensor reading has a hard deadline below 40 ms, choose FEC; for slow trend data choose ARQ. A hybrid (FEC + ARQ) suits worse links.

TASK 9 – How Big Must the Window Be? (4 marks)

QUESTION
Head-office link: 400 km fibre at 1 Gbps, frames 1,500 bytes, light at 2×10^8 m/s. Work out: transmission time, propagation time and a = propagation ÷ transmission; utilisation with stop-and-wait and what it means in Mbps; smallest window for full use and sequence-number bits for Go-Back-N and Selective Repeat; which ARQ resends fewer frames after one loss and its cost; how HDLC/PPP mark frame start and end. **Complete the Code** (TODO 1); smallest window on the satellite link. **Alternative:** Selective Repeat window 64.

Quantity	Calculation	Result
Transmission time t_f	12,000 bits ÷ 10^9 bit/s	**12 µs**
Propagation time t_p	400,000 m ÷ 2×10^8 m/s	**2 ms** (4 ms round trip)
a	2,000 µs ÷ 12 µs	**166.67**

Quantity	Calculation	Result
Stop-and-wait utilisation	1 ÷ (1 + 2a) = 1 ÷ 334.33	**0.299%** → 2.99 Mbps of 1,000 Mbps
Window for 100%	1 + 2a = 334.33	**335 frames** (≈ 502 KB in flight = bandwidth-delay product 4 Mbit)
Go-Back-N sequence bits	window ≤ $2^n - 1$ → 335 ≤ 511	**n = 9 bits**
Selective Repeat sequence bits	window ≤ 2^{n-1} → $2^{n-1} \geq 335$ → 512	**n = 10 bits**

- **After one loss:** Selective Repeat resends only the lost frame (1), Go-Back-N resends the lost frame and all that followed (up to ≈ 335 frames = 502 KB). **Cost of SR:** the receiver needs a buffer for up to 335 frames (335 × 1,500 B = 502 KB), per-frame timers/NAKs and re-ordering logic, and one extra sequence-number bit.
- **HDLC framing:** flag byte 01111110 marks start and end; **bit stuffing** – the sender inserts a 0 after every five consecutive 1s so the flag never appears in data; the receiver removes it. **PPP** (byte-oriented) uses the same flag 0x7E with **byte stuffing**: 0x7E or 0x7D inside data is replaced by 0x7D followed by the byte XOR 0x20 (on synchronous links PPP uses bit stuffing like HDLC).

Completed program (TODOs replaced; file: t9.py)

```
"""Link utilisation of stop-and-wait and sliding-window protocols on different links."""
def utilisation(frame_bits, rate_bps, distance_m, window, speed=2e8):
    tx = frame_bits / rate_bps
    prop = distance_m / speed
    a = prop / tx
    return min(1, window / (1 + 2 * a)), a

links = [("Campus LAN, 200 m, 1 Gbps", 1e9, 200),
         ("City fibre, 50 km, 1 Gbps", 1e9, 50_000),
         ("Satellite, 36,000 km, 10 Mbps", 10e6, 36_000_000)]
for name, rate, dist in links:
    u1, a = utilisation(12_000, rate, dist, 1)
    u7, _ = utilisation(12_000, rate, dist, 7)
    u127, _ = utilisation(12_000, rate, dist, 127)
    print(f"{name:<31} a = {a:7.2f}  stop-and-wait {u1:6.1%}  "
          f"W=7 {u7:6.1%}  W=127 {u127:6.1%}")
u, a = utilisation(12_000, 10e6, 36_000_000, 301)
print(f"Satellite, W=301: {u:.1%}  (smallest full window = 1+2a = {1 + 2 * a:.0f})")
u, a = utilisation(12_000, 1e9, 400_000, 335)
print(f"Head office 400 km: a = {a:.2f}, full window = {1 + 2 * a:.2f} -> 335 frames, U = {u:.0%}")
```

Output obtained (run with Python 3):

```
Campus LAN, 200 m, 1 Gbps       a =    0.08  stop-and-wait  85.7%  W=7 100.0%  W=127 100.0%
City fibre, 50 km, 1 Gbps       a =   20.83  stop-and-wait   2.3%  W=7  16.4%  W=127 100.0%
Satellite, 36,000 km, 10 Mbps   a =  150.00  stop-and-wait   0.3%  W=7   2.3%  W=127  42.2%
Satellite, W=301: 100.0%  (smallest full window = 1+2a = 301)
Head office 400 km: a = 166.67, full window = 334.33 -> 335 frames, U = 100%
```

Satellite link: 1 + 2a = 1 + 2 × 150 = **301 frames** gives 100% utilisation (W = 127 gives only 42.2%).

Alternative – Selective Repeat window 64 on the head-office link

Utilisation = 64 ÷ 334.33 = **19.1%**; throughput = 0.191 × 1 Gbps = **191.4 Mbps** (instead of 1,000 Mbps with window 335). Receiver buffer = 64 × 1,500 B = **96,000 B (96 KB)**; sequence bits 7. A window of 64 wastes 80.9% of the fibre – the window must be about 335.

Evidence to attach: Python output; Packet Tracer ping/WAN test if available.

TASK 10 – Find Where the Delay Comes From (4 marks)

QUESTION

Traceroute: 1 10.10.1.1 0.6/0.5/0.6 ms; 2 100.64.0.1 3.1/2.9/3.4; 3 172.31.4.9 4.0/38.2/4.2; 4 103.21.8.17 11.8/11.5/12.1; 5 * * *; 6 10.200.0.10 12.6/12.4/12.9. Work out: expected propagation delay for 400 km and how much of the measured delay it explains; why hop 3 shows 38.2 ms and why hop 5 shows stars; which hop adds most delay; your own traceroute, ping and Wireshark capture. **Alternative:** SNMP/NetFlow vs occasional ping/traceroute.

- **Propagation:** 400 km ÷ 2×10^8 m/s = **2 ms one way, 4 ms round trip**. The final hop RTT is about 12.6 ms, so propagation explains 4 ÷ 12.6 ≈ **32%**; the other ≈ 8.6 ms comes from the longer-than-straight fibre route, equipment and serialisation, and queueing in the carrier network.
- **Hop 3 (38.2 ms):** a single spike, not a fault – routers generate ICMP Time Exceeded on a low-priority control-plane CPU, so one slow reply says nothing about forwarding delay. Proof: later hops (11.8 ms, 12.6 ms) are lower than 38.2 ms, so the data path itself is not delayed.
- **Hop 5 (* * *):** the router does not answer TTL-expired probes (ICMP filtered or rate-limited) but still forwards traffic – hop 6 answers with normal RTT. Not a fault.
- **Biggest delay:** the step from hop 3 (≈ 4.1 ms median) to hop 4 (≈ 11.8 ms) adds ≈ **7.7 ms** – the long-haul link between 172.31.4.9 and 103.21.8.17 (consistent with the 400 km route, 4 ms propagation plus carrier equipment). Other steps add < 3 ms.

Hop	Median RTT (ms)	Increment (ms)
1	0.6	0.6
2	3.1	2.5
3	4.2	1.1
4	11.8	**7.6**
5	* (no reply)	–
6	12.6	0.8

Own measurements (to be recorded on your machine)

```
ping -c 20 <server>        # record min/avg/max RTT, mdev (jitter) and % loss
traceroute <server>         # (Windows: tracert)  record each hop
Wireshark filter: icmp || dns || tcp   → identify Ethernet / IP / TCP (or ICMP) headers per packet
```

Alternative – monitoring: SNMP counters (polled every 1–5 min; traps instantly) and NetFlow flow records show utilisation, errors, drops and who is using the link, so they would catch real congestion on the 3→4 link and interface errors early (within minutes) and continuously. They would **not** reveal the hop-3 ICMP spike or hop-5 stars (not data-plane problems) and give no path view. Occasional ping/traceroute shows path and RTT at that moment, reveals the stars/spike pattern, but finds problems only when someone runs it (hours/days later) and cannot see intermittent congestion. Best: use both.

Evidence to attach: Own traceroute, ping and Wireshark screenshots with the headers labelled (Task10 folder).

TASK 11 – Defend the Network Design (4 marks)

QUESTION
Recommend the network design for the plant: medium, wireless technology and Ethernet settings per area; error-control and flow-control method per link with the justifying calculation; two biggest remaining risks; whether it works for five plants. Quote numbers from Tasks 1–10. **Alternative:** the whole plant on private 5G instead of wired backbone with Wi-Fi: which traffic on 5G and which must stay on cable?

Area	Medium / technology	Ethernet settings and justification (task)
Robot cell (welders)	Cat6A S/FTP 85 m or fibre; wired	1 Gbps full duplex, auto-negotiation ON both ends (T6 duplex mismatch gave 412 late collisions); SNR 40 dB → 3,322 Mbps vs 15 dB → 1,257 Mbps (T4) so shielding is needed; own VLAN, strict-priority queue (T1: 32 Mbps robot traffic with 10 ms deadline).
Office floor (140 m)	Multimode fibre uplink; Wi-Fi 6 for office users	Cat6 impossible at 140 m (T4); 30 Mbps/line office load (T1).
Shop floor → data centre (550 m)	Single-mode fibre	10G capable; ring of switches with MRP ≤ 200 ms (T6).
Mobile AGVs / moving equipment	Private 5G URLLC (Wi-Fi only as backup)	Wi-Fi: 39 hand-overs × 150 ms = 5.85 s outage vs 5G 3 × 20 ms = 60 ms (T5).
Cameras	Wired, VLAN; motion-triggered recording	96 → 4.8 Mbps per line (T1); jumbo frames only on storage VLAN (T6).
Sensors	Wi-Fi/5G mMTC (fast); LoRaWAN only for slow ones	600 × 50 B/s = 240 kbps ≫ 5 kbps LoRa gateway (T5).
Plant → head office	Leased single-mode line / MPLS, 1–10 Gbps, redundant second path	400 km: 2 ms one way, RTT ≈ 4 ms (T10); 1.27 Gbps demand (T1) > 1 Gbps → 10 Gbps or motion-triggered CCTV (0.54 Gbps).

Link	Error control	Flow control and calculation
Inside LAN (frames)	CRC-32 at Layer 2 (CRC-3 detects all bursts ≤ 3 bits, T7); TCP retransmission above	Sliding window not needed on short links: a = 0.08, stop-and-wait 85.7% (T9).
Head-office WAN	CRC + TCP selective acknowledgements	a = 166.67; stop-and-wait 0.3% (2.99 Mbps); window ≥ 335 frames (≈ 502 KB TCP window); Go-Back-N 9 bits, SR 10 bits (T9).
Radio sensor link	FEC Hamming(21,16): 31.25% overhead, correct in 0 ms (T8)	ARQ costs 40 ms.

- **Risk 1 – WAN:** single provider link (single point of failure; a cut isolates the plant from head office, and one 1 Gbps link is 126.6% loaded at 8 lines). Mitigation: second diverse path, MPLS/VPN backup.
- **Risk 2 – wireless/EMI timing:** 5G hand-over of 20 ms still exceeds the 10 ms deadline (2 lost commands), and welding noise (SNR 15 dB) can reduce wired and wireless margin. Mitigation: dual connectivity, shielded/fibre cabling, safe-stop on the AGV.
- **Five plants:** 5 × 1.27 Gbps = 6.33 Gbps if CCTV streams continuously, or 5 × 0.54 = 2.68 Gbps with motion-triggered CCTV → WAN of 10 Gbps (or 4 × 1G with QoS). Use hierarchical addressing, VLANs per plant and OSPF areas; the design scales.

Alternative – everything on private 5G: 5G should carry the latency-critical and mobile traffic: AGV control (URLLC), robots on moving cells, and sensors (mMTC 240 kbps). Cameras (96 Mbps per line, 768 Mbps for 8 lines) and the back-haul to the data centre should **stay on cable** (eMBB cells would be saturated; fibre is cheaper per Gbps), as should office traffic and the WAN link. Deterministic fixed robots (10 ms) remain on shielded cable, which gives the most reliable delay.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

QUESTION
Changed requirement: a private 5G network on the shop floor and 50 AGVs that move parts between lines and must never miss a command. Re-evaluate the original design (RETAIN → MODIFY → REDESIGN). A. Impact analysis (2): which media, wireless choices and error-control settings are affected; recalculate Task 1 traffic with the AGVs. B. Test the original design (2): can it serve 50 moving AGVs, using hand-over and delay calculations. C. Adaptation or redesign (2). D. Evidence-based defence (2).

A. Impact analysis (2 marks)

Assumption for AGVs (stated so that the numbers can be checked): each AGV has a control command and telemetry of 200 bytes every 10 ms in each direction = 0.16 Mbps + 0.16 Mbps, and one 4 Mbps camera.

Item	Calculation	Traffic
AGV control + telemetry	50 × 0.32 Mbps	16.0 Mbps
AGV cameras	50 × 4 Mbps	200.0 Mbps
AGV total		216.0 Mbps
Plant before (T1)	8 × 158.24	1,265.92 Mbps
Plant with AGVs	1,265.92 + 216	**1,481.92 Mbps ≈ 1.48 Gbps** (148% of a 1 Gbps uplink)
With motion-triggered CCTV (AGV cameras also at 5%)	536.32 + 16 + 10	**562.3 Mbps**

- **Affected:** wireless choice (Wi-Fi 6 → private 5G URLLC for mobile equipment), the shop-floor backbone (≥ 10 Gbps fibre to the 5G core/UPF), the WAN (1 Gbps → 10 Gbps or CCTV reduction), QoS design and error control: radio uses HARQ/FEC (no 40 ms ARQ), duplicate transmission for AGV commands.
- **Not affected:** fixed robot cells (shielded cable stay), office cabling, VLAN plan, CRC/TCP settings on wired links.

B. Test the original design with 50 AGVs (2 marks)

Test	Original (Wi-Fi 6, 30 m APs)	Requirement	Result
Hand-overs per AGV trip	39	–	–
Unreachable time per trip	39 × 150 ms = 5.85 s	≈ 0 (never miss)	**Fail**
Commands missed per trip	585 per AGV	0	**Fail**
50 AGVs on one 600 s cycle	50 × 585 = 29,250 missed commands; a hand-over every 15 s per AGV, i.e. 3.3 hand-overs/s across the fleet	0	**Fail**
With 802.11r (≈ 50 ms)	39 × 50 ms = 1.95 s; 195 missed	0	**Fail**
Bandwidth	1.48 Gbps vs 1 Gbps uplink	≤ 70%	**Fail**

Conclusion: the original design **cannot serve 50 moving AGVs**.

C. Revised design (2 marks) – MODIFY / REDESIGN the wireless part

- **Private 5G URLLC** for the 50 AGVs and mobile robots (4 cells, 3 hand-overs, 60 ms per trip; with dual connectivity ≈ 0 ms); eMBB slice for AGV cameras; mMTC slice for sensors.
- **Fixed robots:** keep shielded Cat6A S/FTP (or fibre) wired, own VLAN with strict priority; **CCTV** on wired VLAN with motion-triggered recording.
- **Backbone:** single-mode fibre ring (MRP) to 10 Gbps; **Wi-Fi 6 for offices** (30 Mbps/line) and guests (separate VLAN); **WAN** 10 Gbps leased line + backup path.
- **Error control:** CRC-32 on wired links, HARQ/FEC on 5G radio, duplicate commands on two radio paths; AGV safe-stop if no command for 20 ms.

D. Evidence-based defence (2 marks)

Criterion	Original	Revised
Unreachable time per AGV trip	5.85 s	60 ms (≈ 0 with dual connectivity)
Missed 10 ms commands per trip	585	6 (≈ 0)
Uplink load	148% of 1 Gbps	≈ 15% of 10 Gbps (1.48 Gbps)

Criterion	Original	Revised
Cost	Lowest	Higher (5G core, radios, 10G optics) – accepted because a missed AGV command risks collision and scrap
Complexity	Low	Higher (5G skills, spectrum, slicing)

Decision: REDESIGN the wireless layer, MODIFY the backbone, RETAIN the wired robot, office and error-control design. The numbers (585 → 6 missed commands, 5.85 s → 60 ms, 148% → 15% uplink load) justify the extra cost; the remaining risk is the 20 ms 5G hand-over, handled by dual connectivity and AGV safe-stop.

Evidence to attach: Packet Tracer/5G design diagram; calculation sheet; GitHub README.md and commit history for Tasks 1–12.